```
"""Command-line interface for Study Organizer.

Examples:
    python cli.py scan ~/Downloads
    python cli.py organize ~/Downloads --dry-run
    python cli.py organize ~/Downloads --recursive --yes
    python cli.py undo ~/Downloads
"""
import argparse
import sys

from organizer.core import (
    DEFAULT_DEST, OrganizerError, execute, plan_moves, summarize,
    undo_last, validate_dest_name, validate_folder,
)


def build_parser():
    parser = argparse.ArgumentParser(
        prog="study-organizer",
        description="Sort study files by course code and file
type.",
    )
    commands = parser.add_subparsers(dest="command",
required=True)

    scan = commands.add_parser("scan", help="summary of what
would happen")
    organize = commands.add_parser("organize", help="move files
into folders")
    undo = commands.add_parser("undo", help="put back the last
organize")

    for sub in (scan, organize, undo):
        sub.add_argument("folder", help="the folder to organize")
        sub.add_argument("--dest", default=DEFAULT_DEST,
                         help=f"output folder name (default:
{DEFAULT_DEST})")
    for sub in (scan, organize):
        sub.add_argument("-r", "--recursive",
action="store_true",
                         help="also look inside sub-folders")
```

```
        sub.add_argument("--no-course", action="store_true",
                         help="sort by file type only, ignore
course codes")

    organize.add_argument("--dry-run", action="store_true",
                          help="list every move without moving
anything")
    organize.add_argument("-y", "--yes", action="store_true",
                          help="do not ask for confirmation")
    return parser


def confirm(question):
    try:
        answer = input(f"{question} [y/N] ")
    except EOFError:
        return False
    return answer.strip().lower() in {"y", "yes"}


def get_plan(args, folder):
    moves = plan_moves(folder, args.dest, by_course=not
args.no_course,
                       recursive=args.recursive)
    if not moves:
        print("No files to organize.")
    return moves


def cmd_scan(args, folder):
    moves = get_plan(args, folder)
    if not moves:
        return 0
    print(f"{len(moves)} file(s) in {folder}\n")
    counts = summarize(moves, folder / args.dest)
    for target, count in sorted(counts.items()):
        print(f"  {count:4d}  ->  {args.dest}/{target}")
    duplicates = sum(m.is_duplicate for m in moves)
    if duplicates:
        print(f"\n{duplicates} duplicate file(s) found.")
    return 0
```

```
def cmd_organize(args, folder):
    moves = get_plan(args, folder)
    if not moves:
        return 0
    for move in moves:
        source = move.source.relative_to(folder)
        target = move.target.relative_to(folder)
        print(f"  {source}  ->  {target}")
    if args.dry_run:
        print(f"\nDry run: {len(moves)} file(s) would be moved. "
              "Nothing was changed.")
        return 0
    if not args.yes and not confirm(f"\nMove {len(moves)}
file(s)?"):
        print("Cancelled. Nothing was changed.")
        return 0

    moved, failed = execute(moves, folder, args.dest)
    print(f"\nMoved {moved} file(s). Use 'undo' to put them
back.")
    for problem in failed:
        print(f"  could not move {problem}", file=sys.stderr)
    return 1 if failed else 0


def cmd_undo(args, folder):
    restored, skipped = undo_last(folder, args.dest)
    print(f"Restored {restored} file(s).")
    for problem in skipped:
        print(f"  skipped {problem}", file=sys.stderr)
    return 1 if skipped else 0


def main(argv=None):
    args = build_parser().parse_args(argv)
    handlers = {"scan": cmd_scan, "organize": cmd_organize,
"undo": cmd_undo}
    try:
        folder = validate_folder(args.folder)
        args.dest = validate_dest_name(args.dest)
        return handlers[args.command](args, folder)
```

```
    except OrganizerError as error:
        print(f"Error: {error}", file=sys.stderr)
        return 1
    except KeyboardInterrupt:
        print("\nStopped by user.", file=sys.stderr)
        return 130


if __name__ == "__main__":
    sys.exit(main())
```